

Python Backend Development Quiz

A short assessment covering core Python concepts, memory management, and asynchronous evaluation.

1. What will be the output of the following Python code execution?

```
def append_to(element, target=[]):  
    target.append(element)  
    return target  
  
my_list_one = append_to(10)  
my_list_two = append_to(20)  
  
print(my_list_two)
```

- A) [20]
- B) [10, 20]
- C) An AttributeError is raised
- D) [10]

2. In Python, how does the Global Interpreter Lock (GIL) primarily impact multi-threaded applications?

- A) It entirely prevents multiple threads from sharing the same memory space.
- B) It forces I/O-bound operations to block all other executing threads synchronously.
- C) It prevents multiple native threads from executing Python bytecodes at once on multiple CPU cores.
- D) It automatically accelerates CPU-bound performance by distribution of workers.

3. Consider the following snippet using dictionary comprehension and scopes. What is the final value of output?

```
x = 5
output = {i: x for i in range(3)}
print(output[1])
```

- A) 1
- B) NameError
- C) 5
- D) 0

4. Which of the following statements accurately describes the difference between `__new__` and `__init__` methods in Python classes?

- A) `__init__` is the actual constructor that creates the instance, while `__new__` initializes it.
- B) `__new__` is a static method responsible for creating and returning a new instance, while `__init__` initializes that instance.
- C) `__new__` is only invoked when inheriting from abstract base classes.
- D) There is no functional difference; `__new__` is simply an obsolete legacy variant of `__init__`.

5. When optimizing database queries in a Python backend ORM (like Django), which technique resolves the "N+1 queries" performance problem for Foreign Key relationships?

- A) Utilizing deferred fields via `.defer()`
- B) Applying `.select_related()` to perform an SQL JOIN operation in the single initial query
- C) Wrapping the queries inside a standard database atomicity block (`transaction.atomic`)
- D) Implementing client-side caching with Redis without changing the query structure

Answer Key & Explanations

Question 1: B

Python's default arguments are evaluated once when the function is defined, not each time it's called. Therefore, the same mutable list instance is reused across multiple invocations.

Question 2: C

The GIL ensures only one thread executes Python bytecode at a time, limiting purely CPU-bound multi-threaded applications from scaling across multi-core systems (though I/O-bound tasks still release the GIL).

Question 3: C

The dictionary comprehension successfully references the variable `x` from the surrounding scope. Each key maps to the value of `x`, which is 5.

Question 4: B

`__new__` is the true allocator/constructor method that returns a clean instance of the class, which is then passed into `__init__` as `self` for attribute assignment.

Question 5: B

`.select_related()` handles forward or backward Foreign Key or OneToOne mappings by performing an SQL JOIN, fetching the related objects immediately so that following loops don't cause separate hits to the database.